

RAG Application

Multi-Tenant Document Q&A System with PDF Chat Capabilities

Application Overview

- RAG application is a multi-tenant web application for chatting with PDF documents
- Users can upload private PDFs and ask questions grounded in document content
- Admins can upload GLOBAL documents visible to all users
- Every conversation is tied to a specific document - no general chat allowed
- ChatGPT-like interface with sidebar showing threads and documents
- Built with Flask backend and server-rendered templates with JavaScript
- Implements RAG (Retrieval Augmented Generation) for accurate responses
- Includes LLM-as-Judge evaluation metrics for response quality assessment

Technology Stack

Core Technologies

- Backend: Flask (Python) - Lightweight WSGI web framework
- Database: Neon/Railway Postgres - Serverless PostgreSQL with SQLAlchemy ORM
- Vector DB: Chroma Cloud - Managed vector database for embeddings
- Object Storage: Backblaze B2 - S3-compatible cloud storage
- LLM: OpenAI GPT-4o - Multimodal language model for chat
- Embeddings: OpenAI text-embedding-3-large - 3072-dimensional vectors
- Auth: Email/Password + GitHub OAuth via Authlib
- Background Jobs: APScheduler for document ingestion

Neon Postgres - Serverless Database

- Serverless PostgreSQL with automatic scaling and branching
- Stores all application data: users, documents, threads, messages
- SQLAlchemy ORM for Pythonic database interactions
- Alembic for database migrations and schema versioning
- Connection pooling with pool_pre_ping for reliability
- Soft-delete pattern using deleted_at timestamps
- UUID primary keys for distributed-friendly IDs
- JSONB columns for flexible metadata storage

Chroma Cloud - Vector Database

- Managed vector database for storing document embeddings
- Stores text chunk embeddings with metadata filtering
- Supports similarity search using cosine distance
- Metadata includes: document_id, page, chunk_id, scope, owner_user_id
- Enables RAG retrieval with top-k similar chunks
- Collection-based organization for multi-tenant isolation
- Automatic embedding dimension handling (3072-dim for text-embedding-3-large)
- Supports deletion by document_id for cleanup operations

Backblaze B2 - Object Storage

- S3-compatible cloud storage for PDF files and extracted images
- Presigned URLs for secure direct uploads from browser
- Path structure: users/<user_id>/documents/<doc_id>/<filename>
- Supports large files up to 100MB (configurable)
- Cost-effective storage with pay-per-use pricing
- boto3 client with path-style addressing
- Pagination support for listing/deleting large object sets
- Automatic cleanup on document deletion

Application Architecture

Project Folder Structure

- app/ - Main application package with Flask factory pattern
- app/models/ - SQLAlchemy database models (User, Document, Thread, etc.)
- app/routes/ - Flask blueprints for API endpoints
- app/services/ - Business logic layer (auth, RAG, storage, vectors)
- app/templates/ - Jinja2 HTML templates for server-side rendering
- app/static/ - CSS, JavaScript, and static assets
- app/workers/ - Background job processors (ingestion worker)
- migrations/ - Alembic database migration scripts
- tests/ - Unit and integration test suite

app/models/ - Database Models

SQLAlchemy models defining the data schema with relationships and methods

User

Authentication model with email/password, GitHub OAuth, role-based access (user/admin)

Document

PDF document metadata with scope (GLOBAL/USER_PRIVATE/ADMIN_ONLY), B2 object key

Thread

Chat conversation tied to a specific document, belongs to a user

Message

Individual chat message (user/assistant role) with JSONB content

IngestionJob

Background job tracking for PDF processing (QUEUED/RUNNING/DONE/FAILED)

ActivityLog

Audit trail for user actions (login, upload, delete, etc.)

MessageEvaluation

LLM-judge evaluation scores per assistant message

app/routes/ - API Endpoints

Flask blueprints organizing REST API endpoints by domain

auth.py

Login, logout, registration, GitHub OAuth callback, session management

documents.py

Presigned URL generation, upload confirmation, document listing/deletion

threads.py

Thread CRUD operations, list threads by document

chat.py

RAG chat endpoint with streaming SSE, quick chat for testing

admin.py

Admin-only endpoints for user/document management

views.py

Server-rendered HTML pages (login, chat interface, admin panel)

app/services/ - Business Logic

Service classes encapsulating core business logic and external integrations

auth_service.py

User authentication, session management, decorators (@login_required, @admin_required)

storage_service.py

Backblaze B2 operations: presigned URLs, upload, download, delete

vector_service.py

Chroma Cloud operations: upsert embeddings, similarity search, delete by document

ingestion_service.py

PDF processing: text extraction, chunking, embedding generation

rag_service.py

RAG pipeline: context retrieval, prompt assembly, LLM chat with streaming

evaluation_service.py

LLM-as-Judge evaluation: faithfulness, citation precision, groundedness

RAG Pipeline & Document Processing

Document Upload Flow

- 1. Client requests presigned URL: POST /api/uploads/presign
- 2. Server validates file size ($\leq 100\text{MB}$) and generates B2 presigned URL
- 3. Client uploads PDF directly to B2 using presigned URL (PUT)
- 4. Client confirms upload: POST /api/documents/confirm
- 5. Server creates Document record and IngestionJob (status=QUEUED)
- 6. Background worker picks up job and processes PDF
- 7. Text extracted page-by-page using PyMuPDF
- 8. Text chunked with configurable size (1000 chars) and overlap (200 chars)
- 9. Embeddings generated via OpenAI text-embedding-3-large
- 10. Vectors upserted to Chroma with metadata

RAG Chat Flow

- 1. User sends message to POST /api/threads/<id>/chat
- 2. Query embedding generated from user message
- 3. Chroma searched for top-k similar chunks (filtered by document_id)
- 4. Retrieved chunks assembled into context with page citations
- 5. System prompt instructs LLM to answer only from provided context
- 6. GPT-4o generates response with streaming (Server-Sent Events)
- 7. Response stored as Message with citations extracted
- 8. LLM-as-Judge evaluates response quality (faithfulness, groundedness)
- 9. Evaluation scores stored in MessageEvaluation table
- 10. Metrics returned to client for display

LLM-as-Judge Evaluation Metrics

Response Quality Evaluation

- LLM-as-Judge approach uses GPT-4o to evaluate response quality
- Three key metrics measured for each assistant response:
 - Faithfulness Score (0.0 - 1.0)
 - Citation Precision Score (0.0 - 1.0)
 - Groundedness Score (0.0 - 1.0)
- Evaluation runs asynchronously after response generation
- Scores stored in MessageEvaluation table for analytics
- Displayed in chat UI metrics panel for transparency
- Helps identify hallucinations and unsupported claims

Faithfulness Score

- Measures how accurately the response reflects the source context
- Evaluates whether claims in the response are supported by retrieved chunks
- Score of 1.0 = All claims directly supported by context
- Score of 0.5 = Some claims supported, some inferred
- Score of 0.0 = Response contradicts or ignores context
- Prompt asks LLM: 'Does the response accurately represent the context?'
- Helps detect hallucinations where LLM invents information
- Critical for document Q&A where accuracy is paramount

Citation Precision Score

- Measures accuracy of page/section citations in the response
- Evaluates whether cited pages actually contain the referenced information
- Score of 1.0 = All citations correctly reference source pages
- Score of 0.5 = Some citations accurate, some incorrect
- Score of 0.0 = Citations are fabricated or wrong
- Regex extracts page numbers from response (e.g., 'Page 14', 'Pages 4 and 7')
- Compares cited pages against retrieved chunk metadata
- Ensures users can verify information in original document

Groundedness Score

- Measures how well the response is grounded in retrieved context
- Evaluates whether response stays within scope of provided information
- Score of 1.0 = Response fully grounded, no external knowledge used
- Score of 0.5 = Partially grounded with some reasonable inferences
- Score of 0.0 = Response relies heavily on external knowledge
- Prompt asks: 'Is the response answerable from the context alone?'
- Penalizes responses that go beyond document content
- Ensures RAG system doesn't fall back to general knowledge

Security & Authorization

Authorization Rules

- Role-based access control: USER and ADMIN roles
- Document scopes: GLOBAL (all users), USER_PRIVATE (owner only), ADMIN_ONLY
- Users can read: GLOBAL docs, their own docs, their own threads/messages
- Admins can: Create GLOBAL/ADMIN_ONLY docs, delete any doc/user
- Admin cannot delete themselves (safety measure)
- Decorators enforce access: @login_required, @admin_required, @owns_document
- Vector search filters by scope and owner_user_id
- Activity logging tracks all sensitive operations

Key Functions Deep Dive

auth_service.py - Key Functions

Authentication and authorization service methods

create_user(email, password, github_id, role)

Creates new user with hashed password, auto-assigns admin role if email matches ADMIN_EMAIL

authenticate_user(email, password)

Validates credentials using bcrypt, returns User or None

login_user(user)

Sets session variables (user_id, email, role), logs activity

get_current_user()

Retrieves User from session, clears session if user deleted

@login_required

Decorator that attaches user to request.user, returns 401 if not authenticated

@owns_document(allow_global)

Decorator verifying document ownership or admin access

rag_service.py - Key Functions

RAG pipeline implementation for document Q&A

generate_query_embedding(query_text)

Creates 3072-dim embedding using OpenAI text-embedding-3-large

retrieve_context(query, doc_id, user_id, top_k)

Searches Chroma for similar chunks, returns text_chunks and citations

assemble_prompt(query, context, history)

Builds messages array with system prompt, context, and conversation history

generate_response(messages, stream)

Calls GPT-4o with assembled prompt, supports streaming via SSE

chat(query, doc_id, user_id, history, stream)

Complete RAG flow: retrieve → assemble → generate, returns response with context

ingestion_service.py - Key Functions

PDF processing and embedding generation

process_document(document_id)

Main entry point: downloads PDF, extracts text, chunks, embeds, upserts to Chroma

extract_text_from_pdf(pdf_path)

Uses PyMuPDF to extract text page-by-page, returns list of (page_num, text)

chunk_text(text, chunk_size, overlap)

Splits text into overlapping chunks for better retrieval

generate_embeddings(chunks)

Batch generates embeddings via OpenAI API

update_job_progress(job_id, progress)

Updates IngestionJob progress (0-100) for UI feedback

vector_service.py - Key Functions

Chroma Cloud vector database operations

upsert_text_embeddings(doc_id, chunks)

Stores text chunk embeddings with metadata (page, chunk_id, scope, owner)

search_text(query_embedding, doc_id, user_id, top_k, is_admin)

Similarity search with permission filtering via _can_access()

delete_by_document(document_id)

Removes all vectors for a document (cleanup on deletion)

_can_access(metadata, user_id, is_admin)

Permission check: GLOBAL=all, USER_PRIVATE=owner/admin, ADMIN_ONLY=admin

_get_collection()

Gets or creates Chroma collection with tenant/database config

storage_service.py - Key Functions

Backblaze B2 object storage operations

generate_presigned_upload_url(object_key, content_type)

Creates time-limited PUT URL for direct browser upload

generate_presigned_download_url(object_key)

Creates time-limited GET URL for secure file access

delete_object(object_key)

Removes single object from B2

delete_objects_by_prefix(prefix)

Paginated deletion of all objects under a prefix (handles >1000 objects)

object_exists(object_key)

HEAD request to check if object exists in bucket

evaluation_service.py - Key Functions

LLM-as-Judge evaluation implementation

evaluate(response, context)

Main entry: calls all three evaluation methods, returns scores dict

evaluate_faithfulness(response, context)

Prompts GPT-4o to rate accuracy of response vs context (0.0-1.0)

evaluate_citation_precision(response, context)

Extracts cited pages, compares to retrieved chunk pages

evaluate_groundedness(response, context)

Prompts GPT-4o to rate if response stays within context scope

_extract_score(llm_response)

Parses numeric score from LLM evaluation response

Data Flow Diagrams

Upload & Ingestion Data Flow

- Browser → POST /api/uploads/presign → Flask validates size → B2 presigned URL
- Browser → PUT to B2 presigned URL → PDF stored in B2
- Browser → POST /api/documents/confirm → Document + IngestionJob created
- APScheduler → Polls QUEUED jobs every 30 seconds
- Worker → Downloads PDF from B2 → Extracts text with PyMuPDF
- Worker → Chunks text (1000 chars, 200 overlap)
- Worker → Generates embeddings via OpenAI API
- Worker → Upserts vectors to Chroma with metadata
- Worker → Updates job status to DONE

Chat Data Flow

- Browser → POST /api/threads/<id>/chat with message
- Flask → Validates thread ownership, gets document_id
- RAG Service → Generates query embedding (OpenAI)
- Vector Service → Searches Chroma for top-k chunks
- RAG Service → Assembles prompt with system message + context
- RAG Service → Streams response from GPT-4o via SSE
- Chat Route → Stores user message and assistant response
- Evaluation Service → Runs LLM-as-Judge evaluation
- Chat Route → Returns response with metrics to browser

Summary & Key Takeaways

Key Takeaways

- RAG application demonstrates production-ready RAG architecture
- Clean separation of concerns: routes → services → models
- Multi-tenant design with role-based access control
- Presigned URL pattern for secure, scalable file uploads
- Streaming responses for better user experience
- LLM-as-Judge evaluation for response quality assurance
- Background job processing for non-blocking PDF ingestion
- Comprehensive activity logging for audit trails
- Soft-delete pattern for data recovery capabilities

Technology Summary

- Flask + Gunicorn: Lightweight, production-ready web server
- Neon Postgres: Serverless database with automatic scaling
- Chroma Cloud: Managed vector DB for semantic search
- Backblaze B2: Cost-effective S3-compatible storage
- OpenAI GPT-4o: State-of-the-art multimodal LLM
- OpenAI Embeddings: High-quality 3072-dim text vectors
- APScheduler: Simple background job processing
- Authlib: OAuth integration for GitHub login

Thank You

RAG Threads - Document Q&A Made Simple